



ARMed and Dangerous

Lo que Graviton5 hace con tus workloads, medido en EKS

Andrés Zeballos · Victor Herrera

Solutions Architect · phData | Cloud Specialist · Caleidos

TRACK

Cloud Architecture & Modernization

SALA

Colaboratorio

HORA

15:05



¿Sabén qué es Graviton?

Levanten la mano. Y ahora: ¿quién ya lo tiene en producción?

[GRAVITON5]

Lo que promete Graviton5

Cifras de AWS al lanzar M9g; las nuestras vienen después.

[CÓMPUTO]

hasta
25 %

más rendimiento de cómputo
que Graviton4 (M8g)

[NÚCLEOS]

192

núcleos en un solo chip

[MEMORIA]

DDR5
8800

"la memoria más rápida de
cualquier instancia en la
nube", según AWS

[CACHE L3]

5×

más L3 que Graviton4 por
chip; por núcleo, 2.6×

Fuente: AWS News Blog, "Now available: Amazon EC2 M9g and M9gd instances...", 10-06-2026; About Amazon (2.6× por núcleo)

Cada año, un Graviton nuevo



[LA PREGUNTA]

¿Por qué no lo usan?

Cuatro dudas que escuchamos cada vez que alguien propone migrar

[ADOPCIÓN HOY]

9 %

de las CPU en clústeres
Kubernetes ya son Arm

y crecen 3.5x más rápido que x86

Instancias cloud en Arm:

9 % → 15 %

en dos años

detrás de las cuatro dudas:
desconocimiento · temor

[PRECIO]

¿De verdad sale más barato?

se escucha: "El ahorro es de marketing"

LO QUE PROBAMOS

Rendimiento por dólar en 6 cargas,
con precios on-demand y Spot

[COMPATIBILIDAD]

¿Mis imágenes y DaemonSets corren en ARM?

se escucha: "Algo se va a romper"

LO QUE PROBAMOS

Un DaemonSet solo amd64 en un
clúster con nodos x86 y Graviton

[ESFUERZO]

¿Cuánto trabajo me cuesta migrar?

se escucha: "Es un proyecto de meses"

LO QUE PROBAMOS

Node groups mixtos y Karpenter
en el mismo clúster

[RENDIMIENTO]

¿Y si mi app rinde menos en ARM?

se escucha: "ARM es para cargas chicas"

LO QUE PROBAMOS

La misma app de m5 a m9g, y 7 cargas
en Graviton5, EPYC y Xeon 6

Fuente: Cast AI, 2026 State of Kubernetes Optimization Report (sus clientes); Datadog, State of Containers and Serverless 2025

[NOMBRES]

¿Qué significa el nombre de un tipo de instancia EC2?

m9g.4xlarge



[LAS TRES DEL LAB]

m8**i**.4xlarge Intel Xeon 6

m8**a**.4xlarge AMD EPYC

m**9g**.4xlarge Graviton5

Fuente: EC2 "Instance type naming conventions" y "General purpose specifications", docs.aws.amazon.com/ec2/latest/instancetypes (29-09-2026)

Siete workloads, cada uno exige algo distinto

Los mismos siete en Intel, AMD y Graviton5; primero, los cinco con SLO de latencia

Java + vthreads

QUÉ ES

Spring REST (PetClinic) en JDK 25, con ajustes y virtual threads

QUÉ EXIGE

muchas peticiones simultáneas: uso de CPU y coordinación entre hilos

CÓMO SE MIDE

peticiones por segundo con el 99 % en < 10 ms

Java tuned

QUÉ ES

Spring REST en JDK 25, con THP y flags de JVM elegidas por chip

QUÉ EXIGE

mucho código: qué tan rápido el núcleo lee y decodifica sus instrucciones

CÓMO SE MIDE

peticiones por segundo con el 99 % en < 10 ms

Go

QUÉ ES

servicio HTTP de la librería estándar

QUÉ EXIGE

cálculo por petición, sin leer datos del disco

CÓMO SE MIDE

peticiones por segundo con el 99 % en < 20 ms

PostgreSQL

QUÉ ES

pgbench select-only, escala 1000 (~17 GB)

QUÉ EXIGE

consultas cortas por índice, con los datos precargados en memoria

CÓMO SE MIDE

consultas por segundo con el 99 % en < 5 ms

MongoDB

QUÉ ES

YCSB workload B (95 % lecturas), 20 M registros

QUÉ EXIGE

lecturas en memoria: qué tan rápido llegan los datos al chip (en caché a propósito)

CÓMO SE MIDE

operaciones por segundo; 99 % de las lecturas en < 5 ms

Y dos más: inferencia y red

Mismos tres chips y mismo clúster; aquí no hay SLO de p99

Inferencia en CPU

QUÉ ES

llama.cpp con Llama 3.1 8B (Q4_0),
4 usuarios generando texto a la vez

QUÉ EXIGE

ancho de banda de memoria y cálculo
vectorial para generar cada token

CÓMO SE MIDE

tokens por segundo entre los 4 usuarios;
no hay SLO de latencia, runs de 6 minutos

Red

QUÉ ES

iperf3 con 8 flujos TCP entre dos
nodos del mismo tipo

QUÉ EXIGE

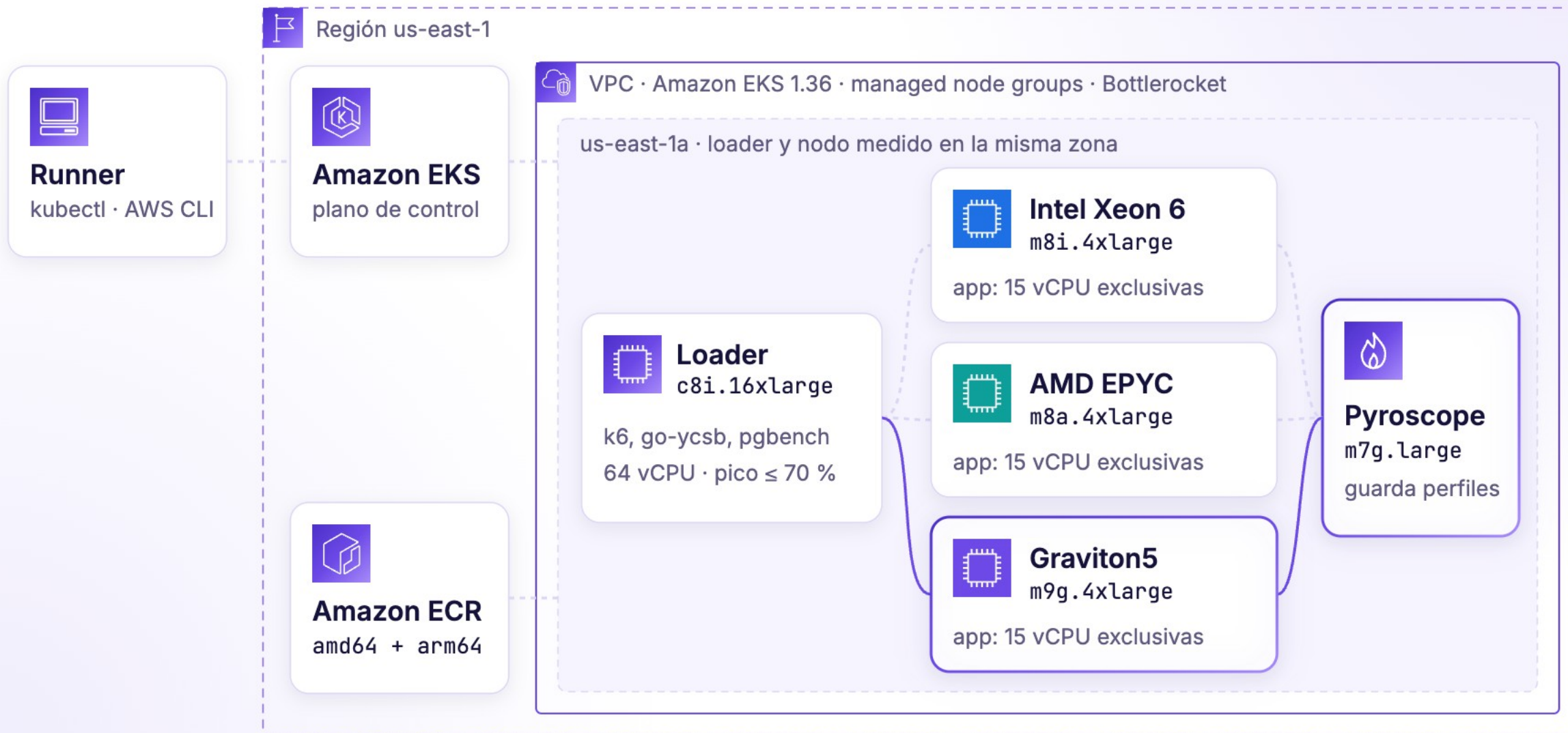
la pila de red del kernel: cuánta CPU
gasta el nodo en recibir tráfico

CÓMO SE MIDE

núcleos de CPU por Gbps recibido;
los Gbps los limita la instancia, no el chip

Mismo clúster, mismo loader, tres procesadores

Una celda a la vez: un nodo medido (dos en red) y perfiles de CPU por eBPF, sin instrumentar la app.



Cómo medimos para comparar los tres

Las reglas para comparar los siete workloads

2 × 3

3 runs de 8 minutos por día, durante 2 días

instancias distintas cada día · inferencia: runs de 6 min · red: un día, 60 s por sentido

15 vCPU

asignadas en exclusiva al pod de la app

el runner verifica las 15 vCPU antes de medir; si faltan, aborta la prueba

≤ 70 %

el loader corre en su propia instancia (c8i.16xlarge)

si pasa del 70 % de CPU, el run se descarta; en red, el tráfico sale de otro nodo igual

punto de quiebre

la mayor carga medida que cumple el SLO de p99

10 ms Java · 20 ms Go · 5 ms PostgreSQL y MongoDB; inferencia y red no tienen SLO

30 / 30

pruebas válidas el día 3

prueba = un workload en un chip con una configuración; ningún run inválido

[VCPU ≠ VCPU]

Mismo tamaño, 16 vCPU, distinto procesador

Qué hay detrás de cada vCPU en una 4xlarge

Intel Xeon 6

m8i.4xlarge

3.9 GHz de turbo sostenido en todos los núcleos



8 núcleos × 2 hilos (SMT) cada par comparte un núcleo físico

AMD EPYC 9R45

m8a.4xlarge

hasta 4.5 GHz



16 núcleos, 1 hilo cada uno

AWS Graviton5

m9g.4xlarge

3.3 GHz

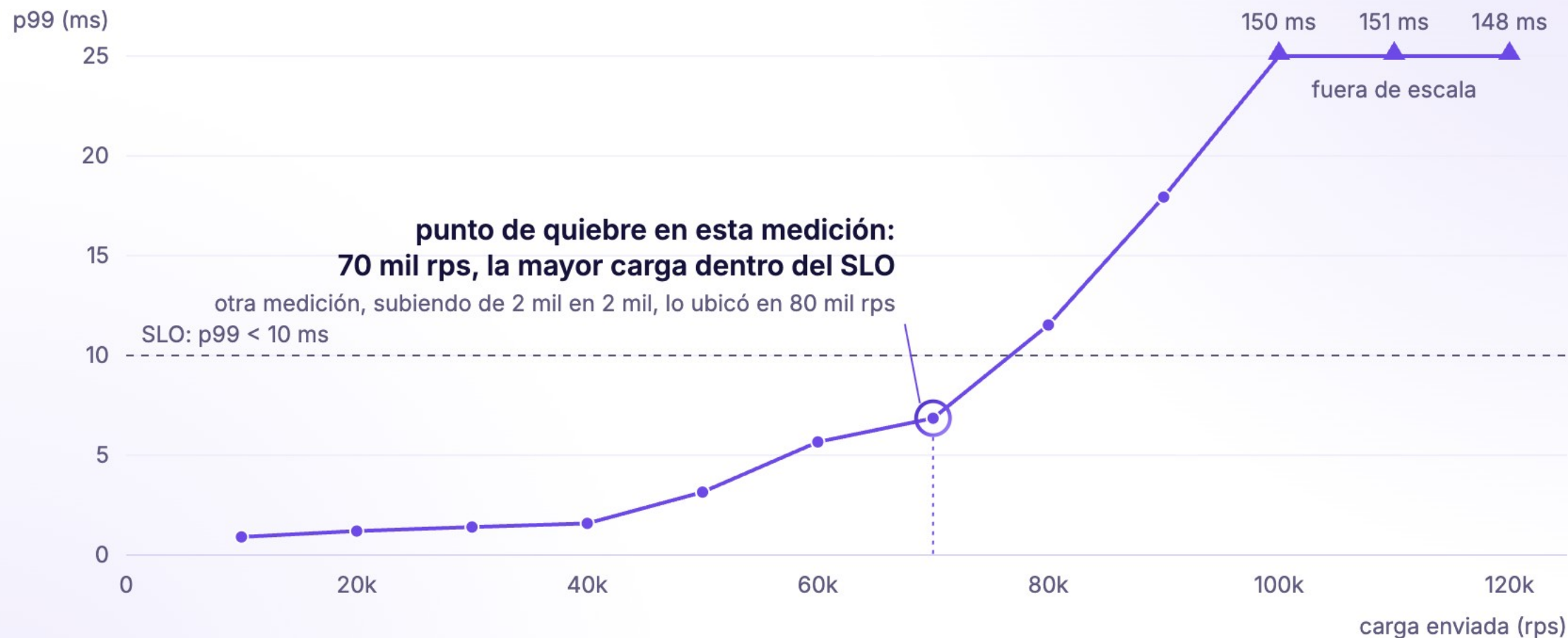


16 núcleos, 1 hilo cada uno

Fuente: EC2 gp.html y describe-instance-types (29-09-2026); páginas de producto M8i y M8a; aws-graviton-getting-started (3.3 GHz)

Qué es el punto de quiebre

Graviton5 (m9g), Java sin ajustes, día 3: el p99 al subir la carga de 10 mil en 10 mil rps



rps / tps / ops/s = peticiones / transacciones / operaciones por segundo

Fuente: lab propio, Task 7 día 3, Java sin ajustes en Graviton5 (results/2026-09-27-task7-d3)

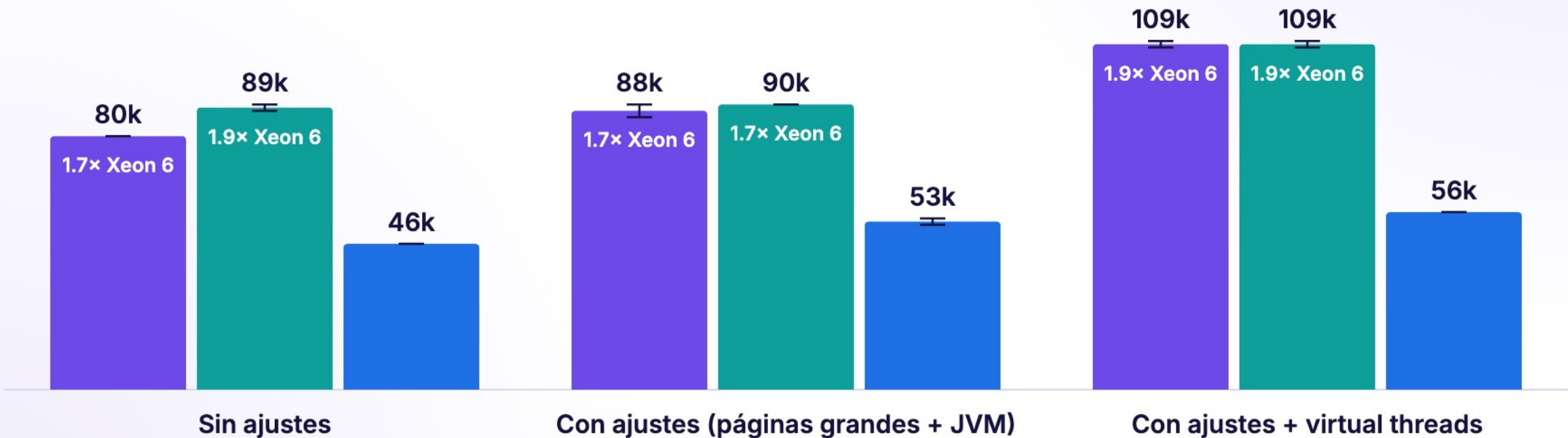
[JAVA]

Java: Graviton5 y EPYC atienden 1.7–2× lo que Xeon 6

Spring REST: peticiones por segundo con el 99 % en menos de 10 ms; más alto = mejor
Sin ajustes, EPYC atiende 11 % más que Graviton5; con ajustes, empatan

■ Graviton5 (m9g) ■ EPYC (m8a) ■ Xeon 6 (m8i)

CONFIG PetClinic REST, base H2 en memoria, heap de 24 GB · k6 sube de 10 mil en 10 mil hasta 120 mil peticiones/s
POR QUÉ EPYC y Graviton5 hacen más trabajo por ciclo (IPC 1.35 y 1.25 contra 0.93) y tienen 16 núcleos reales; Xeon, 8 con SMT



Fuente: lab propio, Task 7, 2 días × 3 runs; barra = media de los 2 días, línea = mín-máx entre días

[G0]

Go: Graviton5 atiende 1.8× lo que Xeon 6

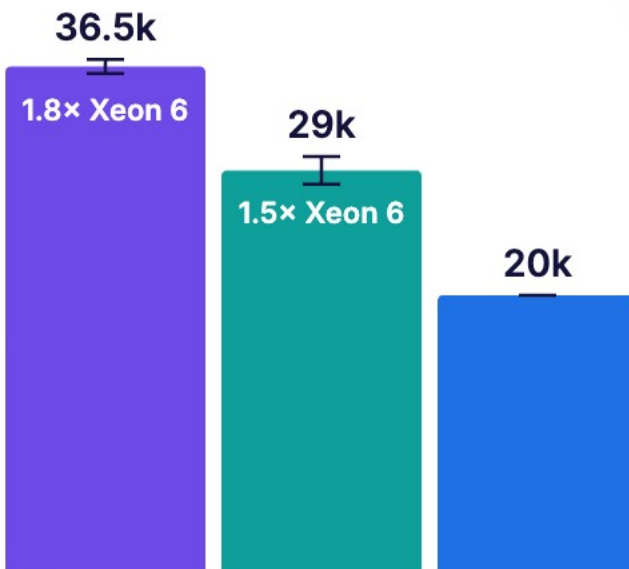
Servicio HTTP: peticiones por segundo con el 99 % en menos de 20 ms; más alto = mejor

Go se midió solo sin ajustes

■ Graviton5 (m9g) ■ EPYC (m8a) ■ Xeon 6 (m8i)

CONFIG servicio HTTP de la librería estándar, compilado genérico (sin flags por chip) · de 5 mil en 5 mil hasta 100 mil peticiones/s

POR QUÉ Graviton5 hace más trabajo por ciclo (IPC 1.23 / 1.02 / 0.68) y falla menos al predecir saltos (1.1 / 2.4 / 3.2 por mil)



Sin ajustes

Fuente: lab propio, Task 7, 2 días × 3 runs; barra = media de los 2 días, línea = mín-máx entre días

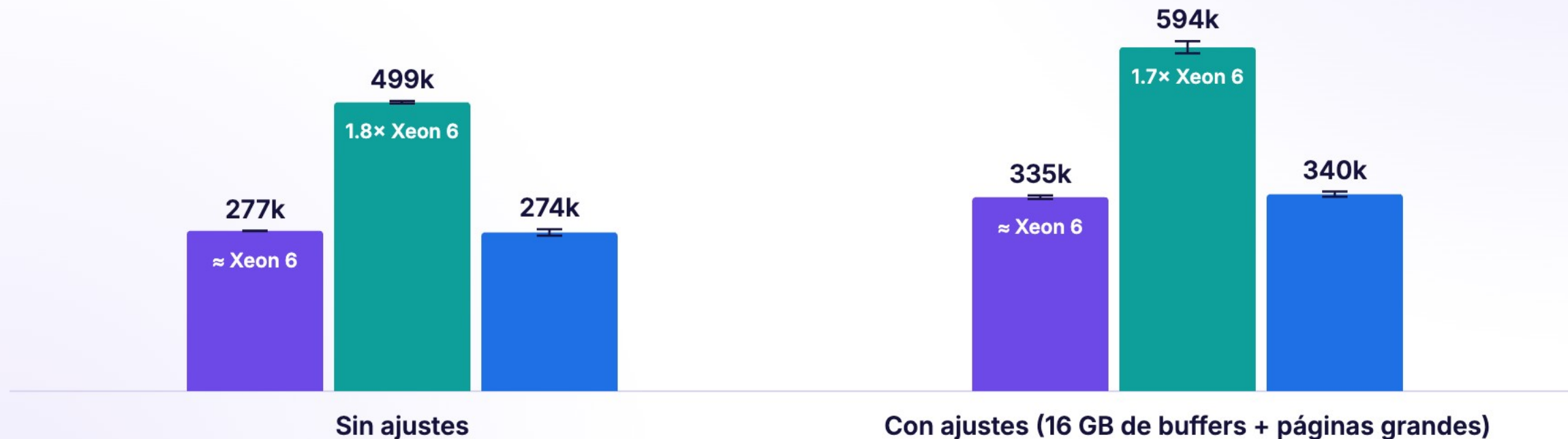
PostgreSQL: aquí gana EPYC; Graviton5 empata con Xeon 6

Transacciones por segundo (una lectura por índice cada una), con el 99 % en menos de 5 ms; más alto = mejor

Con ajustes y precios on-demand, por dólar EPYC rinde 1.52× el Xeon 6; Graviton5, 1.06×

■ Graviton5 (m9g) ■ EPYC (m8a) ■ Xeon 6 (m8i)

CONFIG pgbench solo lectura, 100 M filas (~17 GB) en memoria · de 16 a 512 clientes · quiebre en 128–256 clientes, p99 ≈ 0.6 ms
POR QUÉ EPYC y Graviton5 hacen el mismo trabajo por ciclo (IPC 1.00 y 1.04), pero EPYC va a 4.5 GHz contra 3.3: explica ~1.35× de 1.8×



Fuente: lab propio, Task 7, 2 días × 3 runs; barra = media de los 2 días, línea = mín-máx entre días

MongoDB: Graviton5 atiende 1.3× lo que EPYC y Xeon 6

Operaciones por segundo con el 99 % de las lecturas en menos de 5 ms; más alto = mejor

EPYC y Xeon 6 empatan. En este punto, MongoDB no está limitado por la CPU

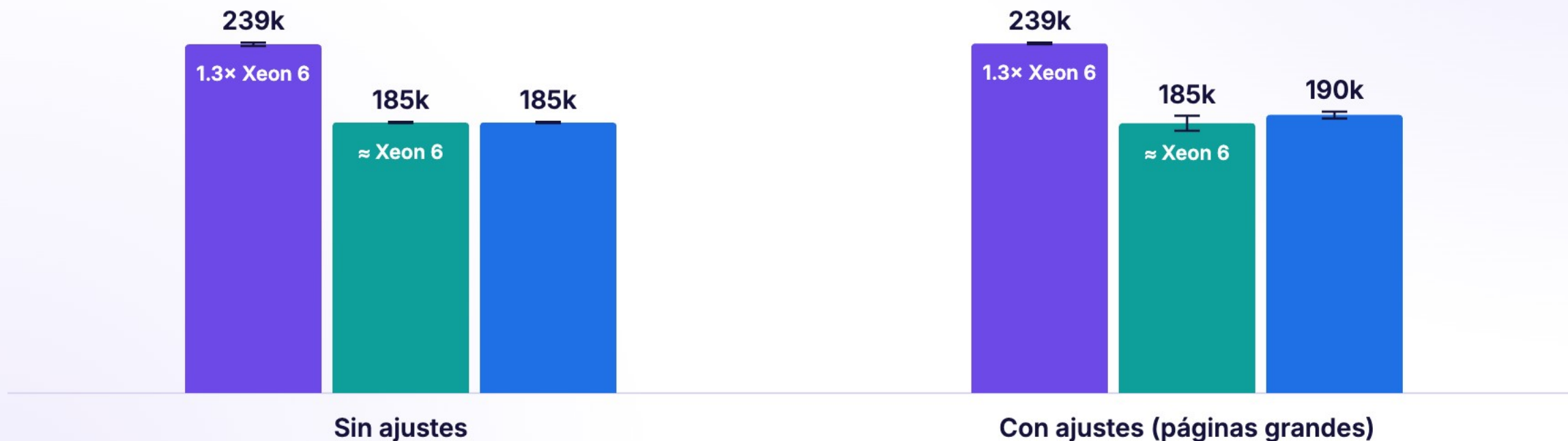
■ Graviton5 (m9g) ■ EPYC (m8a) ■ Xeon 6 (m8i)

CONFIG

YCSB B: 20 M documentos (~20 GB) en caché, 95 % lecturas · de 16 a 512 hilos · quiebre en 128–256 hilos

POR QUÉ

Graviton5 hace más trabajo por ciclo (IPC 1.12 / 0.94 / 0.59), pero aquí la CPU no es el límite: por eso EPYC y Xeon empatan



Fuente: lab propio, Task 7, 2 días × 3 runs; barra = media de los 2 días, línea = mín-máx entre días

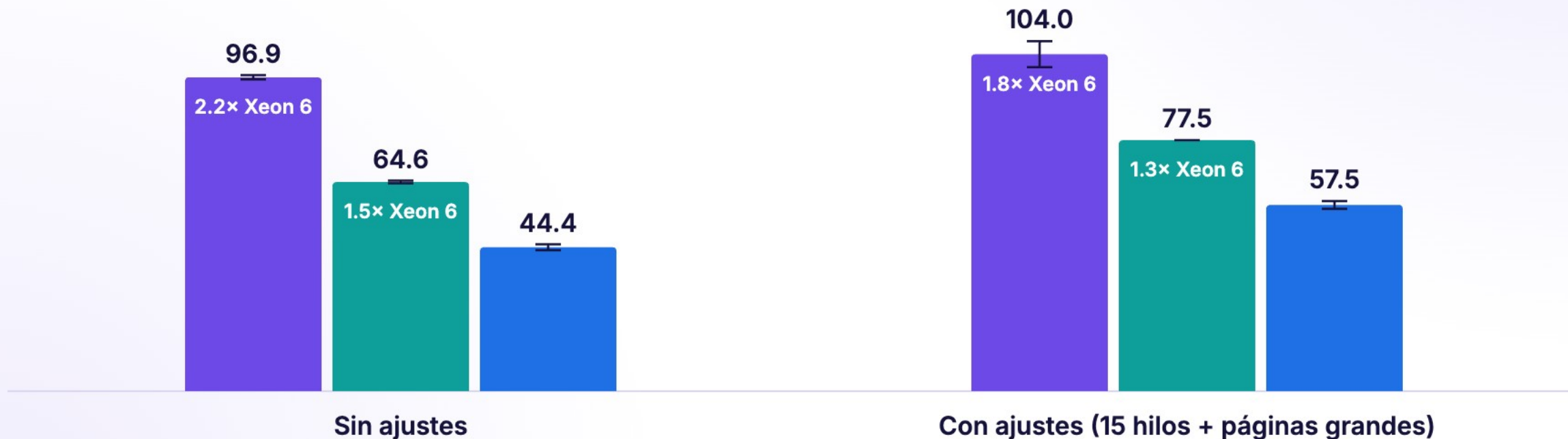
Llama 3.1 8B en CPU: Graviton5 genera ~2× los tokens de Xeon 6

Tokens generados por segundo, sumados entre 4 usuarios a la vez (llama.cpp, Q4_0); más alto = mejor

Con ajustes, un millón de tokens cuesta \$2.09 en Graviton5, \$3.49 en EPYC y \$4.09 en Xeon 6

■ Graviton5 (m9g) ■ EPYC (m8a) ■ Xeon 6 (m8i)

CONFIG modelo de 4.7 GB · 4 usuarios a la vez, respuestas de 128 tokens, mismo prompt · runs de 6 minutos
POR QUÉ sin ajustes, llama.cpp usa 8 hilos en Xeon y 16 en los otros; con 15 hilos, Graviton5 sigue arriba (IPC 1.01 / 0.82 / 0.62)



Fuente: lab propio, Task 7, 2 días × 3 runs de 6 min; barra = media de los 2 días, línea = mín-máx; \$/M tokens: solo el nodo, on-demand

[RED]

Red: Graviton5 gasta un tercio de la CPU de Xeon 6 por Gbps

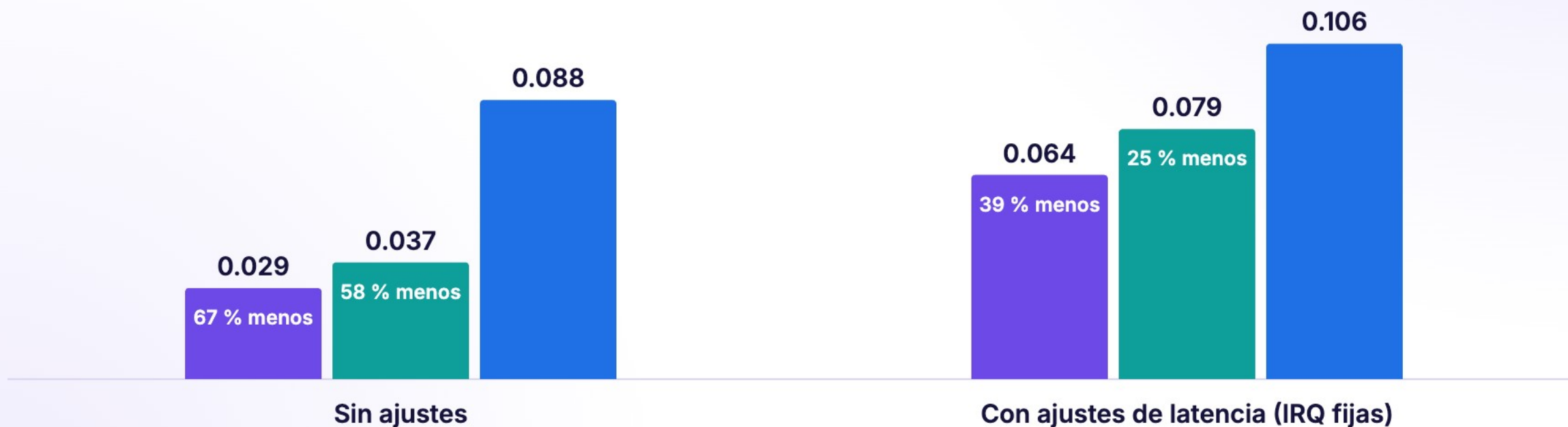
Núcleos de CPU que gasta el nodo por cada Gbps recibido (iperf3, 8 flujos); más bajo = mejor; % frente a Xeon 6

Los Gbps los pone la instancia, no el chip; los ajustes de latencia no sumaron Gbps y sí CPU

■ Graviton5 (m9g) ■ EPYC (m8a) ■ Xeon 6 (m8i)

CONFIG iperf3, 8 flujos TCP, 60 s por sentido, entre dos nodos del mismo tipo · los Gbps los limita la instancia

POR QUÉ Graviton5 y EPYC reciben tráfico a un costo parecido; Xeon paga 2–3×, probablemente porque sus 16 vCPU son 8 núcleos con SMT



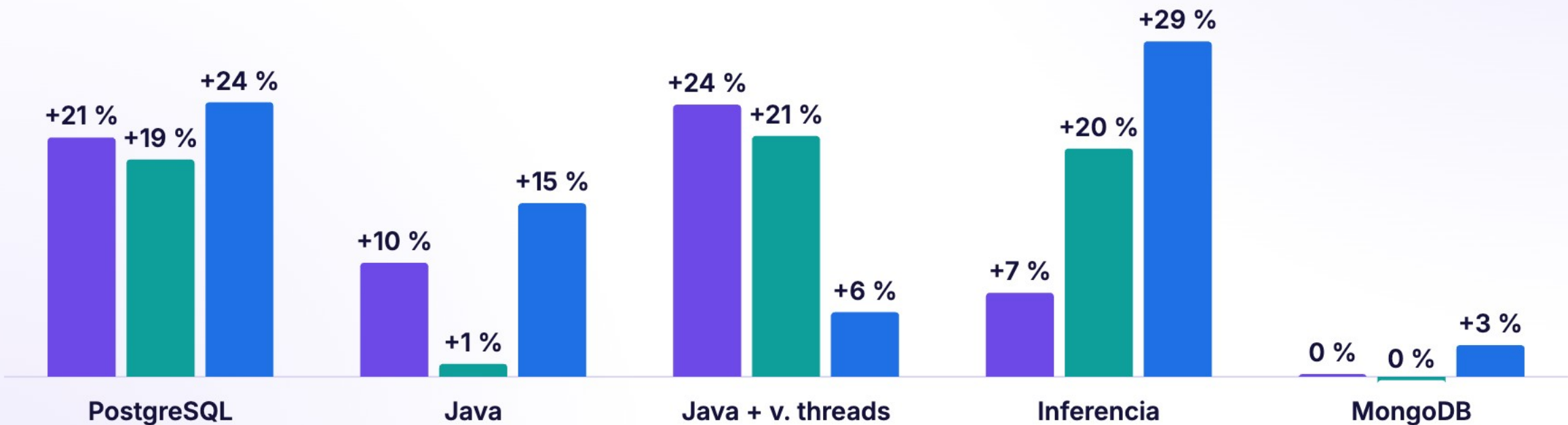
Fuente: lab propio, Task 7 día 2 (un solo día, 3 runs); throughput 16.9 Gbps m9g, 14.9 m8a y m8i

Ajustar paga, pero no igual en cada chip

Ganancia frente a la versión sin ajustes; Java + virtual threads, frente a Java con ajustes; más alto = mejor
En red (1 día), los ajustes de latencia costaron entre 20 % y 124 % más CPU por Gbps recibido

Graviton5 (m9g) EPYC (m8a) Xeon 6 (m8i)

CONFIG PostgreSQL: 16 GB de buffers + páginas grandes · Java: páginas grandes + flags de JVM · inferencia: 15 hilos · MongoDB: páginas grandes
POR QUÉ ajustar rinde donde el default te deja lejos: sin -t, llama.cpp usa 8 hilos en Xeon (+29 %); MongoDB no estuvo limitada por CPU



Fuente: lab propio, Task 7, media de 2 días; dentro del ruido: Java en EPYC, MongoDB en Graviton5 y EPYC; inferencia en Graviton5: leve (0–13 %)

[¿PRECIO?]

¿Precio? Más trabajo por cada dólar

Graviton5 (m9g) frente a Xeon 6 (m8i), mismo tamaño .4xlarge, on-demand us-east-1

~2x

por dólar en Go, Java e inferencia:
cerca de la mitad del costo por unidad de trabajo

m9g cuesta 7.6 % menos por hora que m8i y además rinde más

1.97x

Go
sin ajustes

2.11x

Java
con ajustes + virtual threads (1.80x sin ellos)

1.96x

Inferencia
con ajustes; 2.36x sin ajustes

1.36x

MongoDB
con ajustes

1.06x

PostgreSQL
aquí gana EPYC: 1.52x por dólar

1.23x

Red
Gbps por dólar; tope de la instancia, 1 día

En Spot, EPYC supera a Graviton5 en Java sin ajustes

Mediana semanal us-east-1: Java sin ajustes, EPYC 1.99x contra Graviton5 1.86x; Go, inferencia y MongoDB siguen con Graviton5

Fuente: medido en el lab, Task 7, 2 días x 3 runs (red: 1 día); precios on-demand y Spot: AWS Pricing API y EC2 API, us-east-1

Por dólar, Graviton5 lidera en seis de siete escenarios

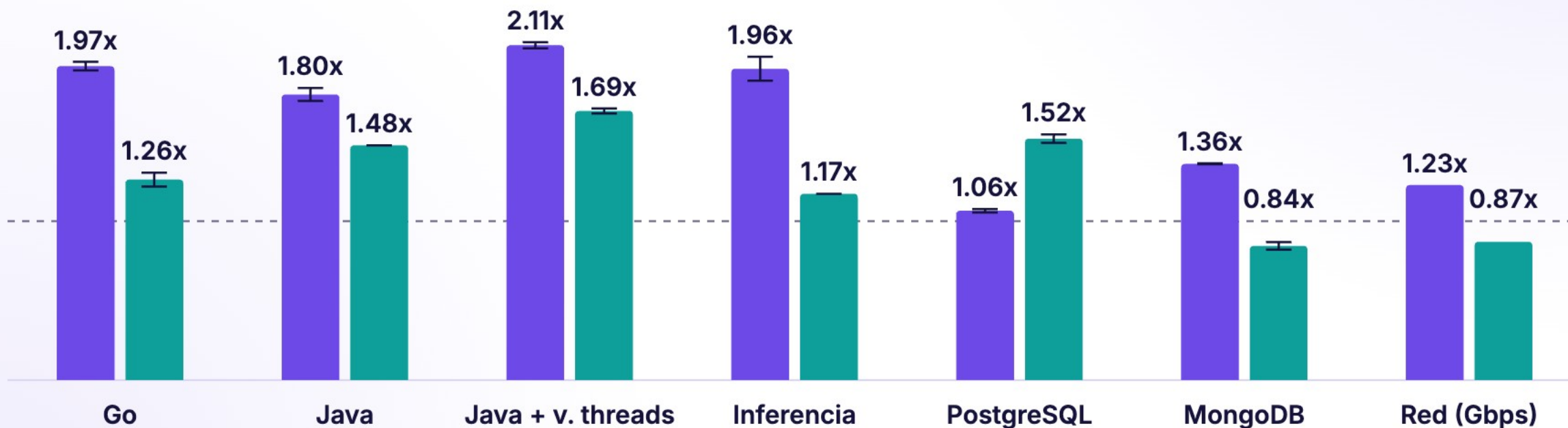
Cuánto trabajo da cada dólar por hora frente a Xeon 6 (1.00x, línea punteada); más alto = mejor

Misma variante en los tres chips: con ajustes, salvo Go y red; red = Gbps por dólar, un solo día

■ Graviton5 (m9g) ■ EPYC (m8a) - - - 1.00x = Xeon 6 (m8i)

CONFIG precio por hora, on-demand us-east-1: Graviton5 \$0.78 · Xeon 6 \$0.85 · EPYC \$0.97 (mismo tamaño .4xlarge)

POR QUÉ Graviton5 cuesta 8 % menos que Xeon 6 y 20 % menos que EPYC, y en estas pruebas rinde igual o más; en PostgreSQL, EPYC compensa su precio



Fuente: lab propio, Task 7, 2 días × 3 runs; barra = media de los 2 días, línea = mín-máx entre días; precios: AWS Pricing API

[¿RENDIMIENTO?]

¿Rendimiento? La misma app, desde un m5

Si hoy corres Java en un m5: cuánto ganas al moverte (Java sin ajustes, 1 run por familia: orientativo)

m5 → m9g

3.3x

Graviton5, la misma app sin tocar:
24 mil → 80 mil rps; 3.27x por dólar

3.75x

m5 → m8a (EPYC)

el salto más grande en crudo
2.96x por \$

2.0x

m5 → m8i (Xeon 6)

el salto en Xeon, sigue en x86
1.81x por \$

1.67x

m5 → m8g (Graviton4)

la generación anterior de Graviton
1.78x por \$

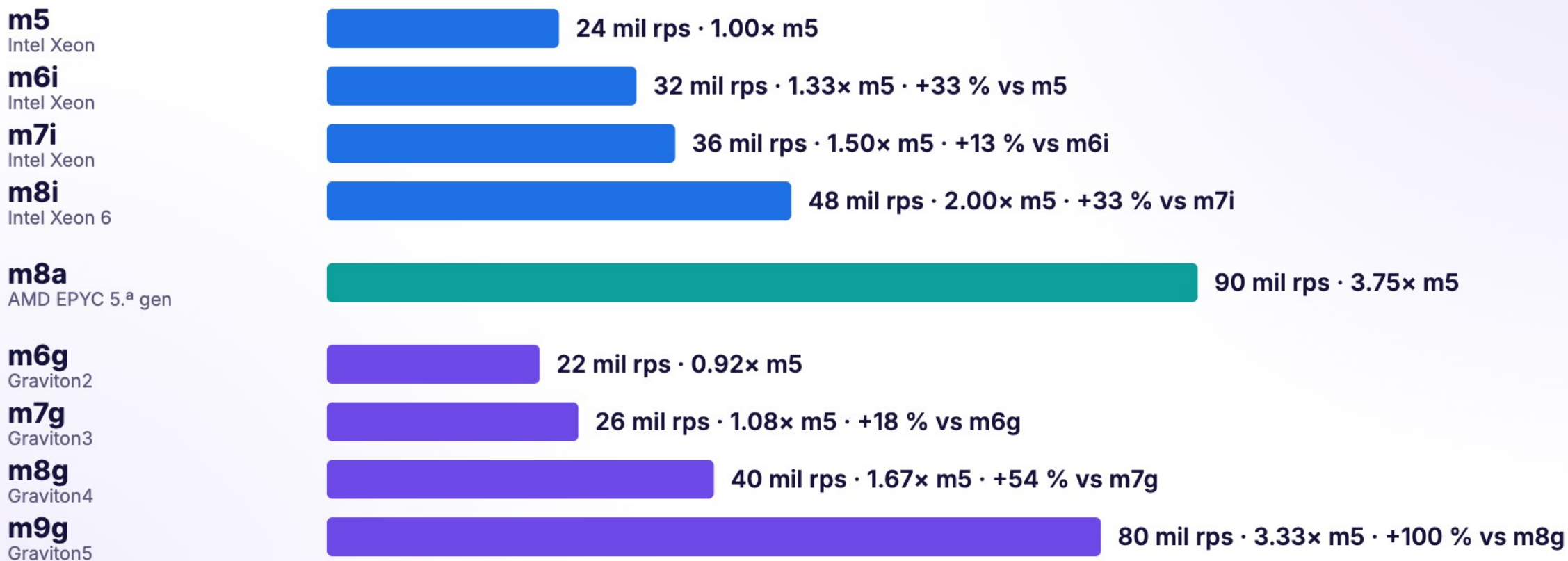
Graviton5 frente a Graviton4 (m8g): 2x en nuestra carga, ~30 % en suites amplias

Medido: 80 mil contra 40 mil rps, repetido en 2 nodos m8g. Terceros: Phoronix +30 % de media geométrica (140+ pruebas);
Honeycomb, 11-26 % menos CPU en producción. Decide tu carga.

Fuente: medido en el lab, Task 8 (arco, 2026-09-28, 1 run por familia); terceros: Phoronix 2026-07-09, Honeycomb 2026-06-10

De Graviton4 a Graviton5, la misma app hizo el doble

Java sin ajustes: peticiones por segundo en el punto de quiebre de cada familia .4xlarge; más largo = mejor
Intel creció 2x en cuatro generaciones (m5 → m8i); Graviton, 2x en una sola (m8g → m9g)

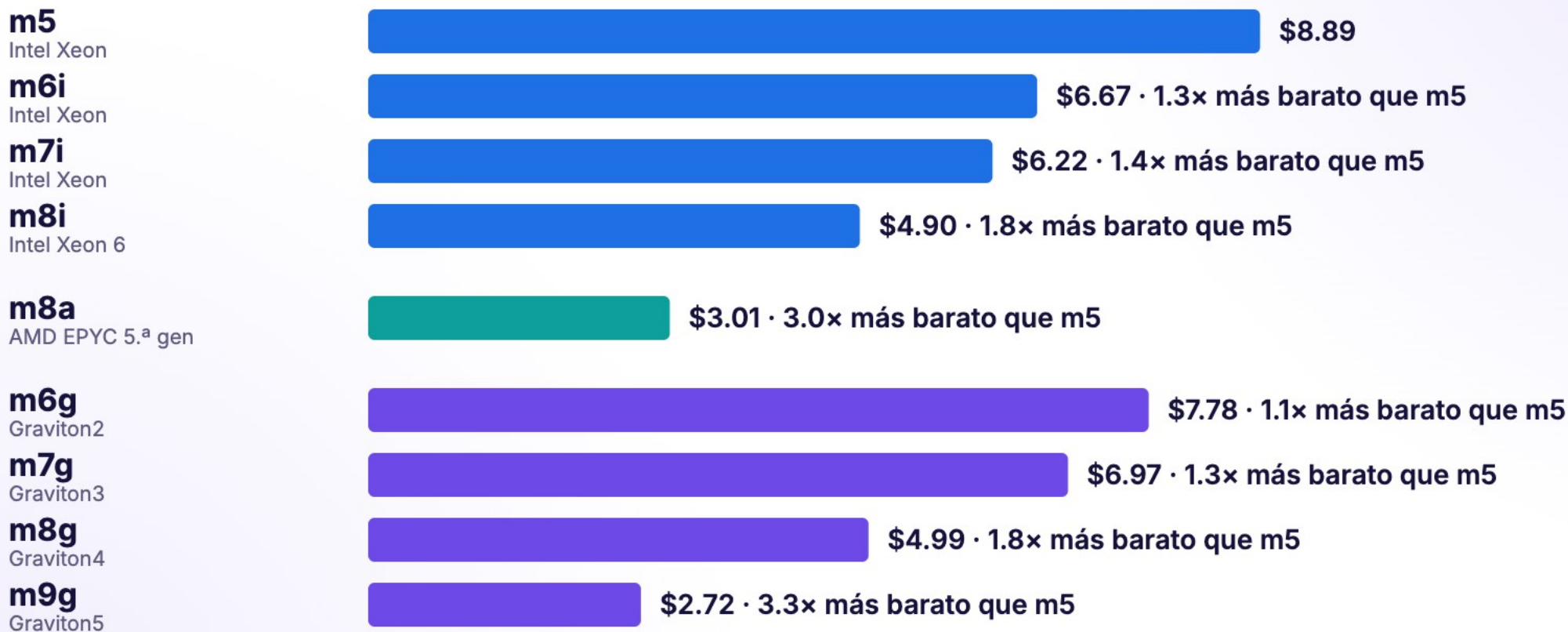


Fuente: lab propio, Task 8, 1 run por familia (m8g: 2 instancias, 40 mil rps cada una); orientativo

Con Java, cada petición en m9g cuesta un tercio que en m5

Costo del nodo por cada mil millones de peticiones, al punto de quiebre, on-demand us-east-1; más corto = mejor

Si debes seguir en x86: m8a cuesta un tercio (2.96× por \$); m8i, poco más de la mitad (1.81×)



Fuente: lab propio, Task 8, 1 run por familia; precios: AWS Pricing API; orientativo

[¿COMPATIBILIDAD?]

¿Compatibilidad? Lo que se rompe está fuera de tu app

Un DaemonSet con imagen solo amd64, en un clúster con nodos x86 y Graviton

```
$ kubectl apply -f legacy-agent-daemonset.yaml
$ kubectl get pods -l app=legacy-agent
NAME                READY  STATUS   RESTARTS   #   arm64 · m6g
legacy-agent-6xcqj  0/1    Error    3          #   arm64 · m6g
legacy-agent-8tg1w  1/1    Running  0          #   amd64 · m6i
legacy-agent-d62ht  0/1    Error    3          #   arm64 · m7g
$ kubectl logs legacy-agent-6xcqj
exec /bin/sh: exec format error
```

La imagen se descargó sin error;
el fallo apareció al arrancar el contenedor.

[EN ESTA ESCENA]

Agentes y DaemonSets

monitoreo, seguridad, logs: si uno
es solo amd64, falla en cada nodo arm

[EN EL LAB]

El generador de carga

construimos go-ycsb solo para amd64:
el loader se quedó en x86

[OPCIONAL]

Optimizaciones por chip

para activar KleidiAI compilamos otra
imagen; en calibración, sin ganancia

Antes de migrar `docker buildx imagetools inspect <imagen>`

lista las plataformas de cada imagen

Fuente: escena medida en el lab, Task 9, 2026-09-28 (salida real de kubectl, columnas acortadas); buildx: Docker Docs

[¿ESFUERZO?]

¿Esfuerzo? De x86 a ARM, una línea por manifiesto

Con imágenes multi-arch listas, solo cambió el nodeSelector en Java, Go, PostgreSQL y MongoDB

```
$ diff overlays/x86-stock/ overlays/arm-stock/
spec:
  nodeSelector:
-   aad/cell: x86-stock
+   aad/cell: arm-stock
# misma etiqueta de imagen, mismos recursos
```

[DÓNDE ESTÁ EL TRABAJO]

Antes: imágenes multi-arch y dependencias revisadas

Después: mover un servicio, medir su punto de quiebre
y seguir con el siguiente.

[LO QUE HICIMOS]

Una imagen para los dos

buildx --platform linux/amd64,linux/arm64;
Postgres, Mongo y llama.cpp ya vienen así

[LO QUE NO TOCAMOS]

El código de las apps

Java: mismo JAR.

Go: un binario por arquitectura, mismo código

[LO QUE SÍ CAMBIA]

Los ajustes finos

los flags de JVM se eligen por chip;
sin ajustes, solo cambia el nodeSelector

Node groups x86 y arm en el mismo clúster: migras servicio por servicio, sin big-bang

Fuente: manifiestos del lab (overlays x86-stock y arm-stock); apps/build-multiarch.sh (buildx linux/amd64,linux/arm64)

[¿Y FARGATE?]

¿Y si corro en Fargate? No eliges la generación

Lo medido aquí es EC2. En ECS Fargate eliges x86 o Arm, pero AWS elige la generación

[EKS + FARGATE]

Sin Arm

La tabla de EKS dice: «Arm processors: No».
Tampoco DaemonSets ni Fargate Spot.
Para Graviton en EKS: nodos EC2,
como en este lab.

[ECS + FARGATE]

Arm sí: cpuArchitecture ARM64

20 % menos por vCPU y GB que x86;
AWS dice «hasta 40 %» mejor precio-
rendimiento (cifra de AWS, no medida aquí).
Spot con Graviton: sí.

[EL HARDWARE ES DINÁMICO · GITHUB [aws/containers-roadmap #2230](#)]

AWS anunció la incorporación de Graviton3 a Fargate en 2024: «We added Graviton 3 to Fargate fleet»

En pruebas pequeñas de octubre de 2024, un usuario observó Graviton2 y Graviton3 en ECS Fargate
Un usuario identificó la instancia con `cat /sys/class/dmi/id/product_name` (sin garantía de AWS)

¿Necesitas elegir generación? EKS con nodos EC2 o ECS Managed Instances (tipo y fabricante)

Fuente: docs de EKS y ECS sobre Fargate, FAQ y precios de Fargate (30-09-2026); GitHub [aws/containers-roadmap #2230](#), [#1030](#), [#2019](#)

Lo que medimos en los tres chips

Mismo tamaño .4xlarge, 16 vCPU, siete workloads; precios on-demand us-east-1

[GRAVITON5]

~2×

por dólar frente a Xeon 6

en Go, Java e inferencia; lidera por dólar en 6 de 7 escenarios (media geom., 5 clases: 1.59×

[EPYC]

1.75×

Xeon 6 en PostgreSQL con ajustes

el más rápido en PostgreSQL y en Java sin ajustes; en promedio, similar a Graviton5

[XEON 6]

8 + SMT

contra 16 núcleos físicos

mismas 16 vCPU, la mitad de núcleos: último o empatado en los siete workloads

El chip conveniente depende de tu carga: mide tu punto de quiebre.

Fuente: medido en el lab, Task 7, 2 días × 3 runs (red: 1 día); precios on-demand us-east-1 (AWS Pricing API)

Qué haría según tu punto de partida

Rutas de la guía de decisión; las cifras del arco son Java sin ajustes frente a m5.4xlarge, mismo JDK

01

Tienes que seguir en x86 (por el proveedor o la licencia)

m8a; y si puedes, JDK 17+ antes de moverte

3.75× m5

[Java · arco]

02

Java o Go con imágenes multi-arch

m9g, con la familia fijada en Karpenter (si no, eligió m6g)

3.3× m5

[Java · arco]

03

PostgreSQL

m8a + shared_buffers de 16 GB y páginas grandes (+19 %)

1.75× Xeon 6

[ambos con ajustes]

04

Generación de tokens en CPU

m9g, y fija los hilos (-t) al tamaño del pod

\$2.09 / M tokens

[Llama 3.1 8B · con ajustes]

05

Antes de migrar

imágenes multi-arch, DaemonSets y agentes incluidos

exec format error

[escena]

Fuente: guía de decisión del lab; Task 7 (2 días × 3 runs) y Task 8 (arco, 1 run por familia: orientativo); \$/M tokens: solo el nodo, on-demand

Lo que aprendimos midiendo CPUs en Kubernetes

Sirve para cualquier benchmark que hagas en tu clúster



El loader es el primer sospechoso

si pasa del 70 % de CPU, el run no cuenta; un cliente YCSB topaba en 191 mil ops/s, dos llegaron a 242 mil



«Sin ajustes» no es neutral: los defaults ya eligen por ti

llama.cpp sin -t usó 8 hilos en el Xeon y 16 en Graviton5 y EPYC



Revisa el disco antes de culpar al chip

MongoDB en gp3 de 125 MiB/s dio 134 mil de 194 mil ops/s; con 1000 MiB/s, los runs se sostuvieron



Mide la perilla en tu carga

la de red de un runbook de latencia costó 2.2× CPU por Gbps en Graviton5 (1 día)



El kernel también es parte del benchmark

Phoronix: Linux 7.0 mejoró PostgreSQL (lectura/escritura) en otro EPYC frente a 6.19; aquí: 6.18, solo lecturas



Define el punto de quiebre antes de mirar los datos

el mayor rendimiento que cumple el SLO de p99: 10 ms Java, 20 ms Go, 5 ms las bases



Dos instancias en días distintos deciden los empates

de un día a otro, 0-8 %; Java con ajustes m9g contra m8a: 0.98×, dentro del ruido

Fuente: medido en el lab (Task 7 y calibración; guardas del runner); kernel: Phoronix, «Linux 7.0 ... PostgreSQL ... AMD EPYC»

Python y MySQL: pendientes de medir

Lo que queremos averiguar y lo que ya sabemos

Python

no medido todavía

¿Qué chip rinde mejor con Python?

¿Todas tus dependencias nativas tienen paquetes compilados para arm64?

LO QUE YA SABEMOS

Phoronix: m9g superó a m8g
en PyPerformance (terceros)

MySQL

no medido todavía

¿EPYC también lidera en MySQL?

¿O su ventaja era de PostgreSQL
y no de las bases relacionales?

LO QUE YA SABEMOS

PostgreSQL con ajustes: EPYC 1.75×
Xeon 6; Graviton5 y Xeon 6 empataron

[PARA LLEVAR]

Repite el lab en tu carga

Todo está en el repo: código, datos crudos, análisis y fuentes



github.com/andrezc98/armed-and-dangerous



El repo del lab

Código, manifiestos y datos de cada run
github.com/andrezc98/armed-and-dangerous



La guía de decisión

Rutas y mediciones por stack: «corro X hoy, ¿qué gano?»
[analysis/decision-guide.md](https://github.com/andrezc98/armed-and-dangerous/blob/main/analysis/decision-guide.md)



Las fuentes verificadas

Fuentes y alcance de cada cifra de esta charla
[slides/fuentes.md](https://github.com/andrezc98/armed-and-dangerous/blob/main/slides/fuentes.md)

[EL LUNES]

1

Revisa tus imágenes

amd64 y arm64 en cada una

2

Agrega un node group arm64

m9g junto al grupo x86

3

Mide tu punto de quiebre

tu SLO en ambas instancias



¿Preguntas?

Levanta la mano o acércate al terminar la charla

[TU OPINIÓN]

¿Qué te pareció la charla?

Escanea el código y déjanos tu feedback: nos ayuda a mejorar la próxima



ARMed and Dangerous: lo que Graviton5 hace con tus workloads,
medido en EKS

escanea para calificar esta sesión



Comparte tu experiencia

Sube tus fotos del día y etiqueta a la comunidad

 **Instagram** @aws.community.day.peru

 **LinkedIn** AWS Community Day Perú

 **TikTok** @aws.community.day.peru

 **X** @AWS_ComDay_Peru

 awscommunityday.pe



¡Gracias!

Andrés Zeballos · Victor Herrera

Solutions Architect · phData | Cloud Specialist · Caleidos

Andrés: [in/andreszc](#) · [github.com/andrezc98](#)

Victor: [in/victor-herreraa](#) · [github.com/VotirchH](#)



ESCANEA Y CONECTEMOS